37. Compose and Kubernetes Concepts for Operators

37. Compose and Kubernetes Concepts for Operators

Phase: Phase 11 — Containers and cloud Linux
Estimated time: 2 days
Slug: `37-compose-k8s-lite`

Learning objectives

- Read a docker-compose file
- Map pods/services/deployments mentally
- Kubectl get/describe/logs basics
- Know what Linux skills transfer to k8s nodes

Prerequisites

Complete prior numbered lessons. Prefer a real Linux VM/container for Linux-specific topics. Type every command yourself — muscle memory beats skimming.

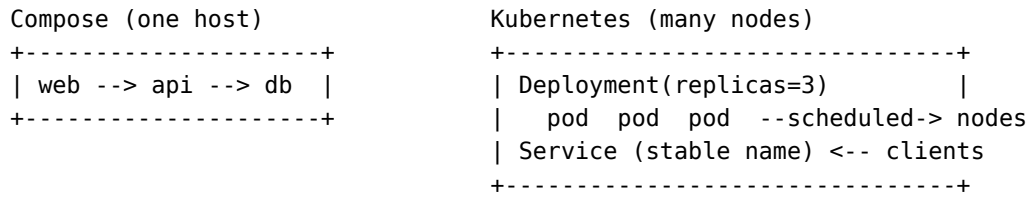
Why this matters

Once you run more than one container, typing `docker run` by hand stops scaling. **Docker Compose** describes a multi-container application in one YAML file on a single host. **Kubernetes** (k8s) goes further: it schedules containers ("pods") across many machines and keeps them running. You do not need to be a cluster administrator to benefit; you do need the basic vocabulary and the handful of `kubectl` commands that answer "what is running and why is it broken?" The most used of these is `kubectl logs`.

Core concepts

Compose term	Meaning
service	One container definition (image, ports, env)
network	Private network where services find each other by name
volume	Persistent data
`depends_on`	Start order hint (not readiness)

Kubernetes term	Meaning
pod	Smallest unit: one or more containers sharing network and storage
deployment	Declares desired replicas of a pod; handles rolling updates
service	Stable virtual IP/DNS name in front of pods
namespace	Logical partition of cluster resources
configmap/secret	Configuration and sensitive data injected into pods
node	A machine that runs pods
ingress	HTTP routing into the cluster



Kubernetes is **declarative**: you apply desired state and controllers keep reconciling reality toward it. If a pod dies, the deployment creates another.

Worked examples

A Compose file, `compose.yaml`:

```yaml
services:
  web:
    image: nginx:1.27
    ports: ["8080:80"]
    depends_on: [api]
  api:
    image: myapp:0.1
    environment:
      DB_HOST: db
    depends_on: [db]
  db:
    image: postgres:16
    environment:
      POSTGRES_PASSWORD: example
```

```
    volumes: ["pgdata:/var/lib/postgresql/data"]
volumes:
  pgdata:
```

```
docker compose up -d
docker compose ps
docker compose logs -f api
docker compose exec db psql -U postgres -c 'select 1'
docker compose down            # keep volumes
docker compose down -v         # also delete volumes
```

Kubernetes manifest, `app.yaml`:

```
apiVersion: apps/v1
kind: Deployment
metadata: {name: hello}
spec:
  replicas: 2
  selector: {matchLabels: {app: hello}}
  template:
    metadata: {labels: {app: hello}}
    spec:
      containers:
        - name: hello
          image: nginx:1.27
          ports: [{containerPort: 80}]
---
apiVersion: v1
kind: Service
metadata: {name: hello}
spec:
  selector: {app: hello}
  ports: [{port: 80, targetPort: 80}]
```

Daily kubectl loop (try with kind or minikube):

```
kubectl apply -f app.yaml
kubectl get pods -o wide
kubectl describe pod POD
kubectl logs POD
kubectl logs deploy/hello --tail=50 -f
kubectl logs POD -c CONTAINER --previous
kubectl exec -it POD -- sh
kubectl get events --sort-by=.lastTimestamp
kubectl rollout status deploy/hello
kubectl scale deploy/hello --replicas=4
kubectl delete -f app.yaml
```

Reading pod status:

Status	Usual meaning	First command
Pending	cannot be scheduled / image pulling	`describe pod`
ImagePullBackOff	wrong image name or no credentials	`describe pod`
CrashLoopBackOff	process exits repeatedly	`logs --previous`
Running but not Ready	readiness probe failing	`describe`, `logs`
OOMKilled	memory limit exceeded	`describe` (Last State)

Local clusters:

```
kind create cluster
kubectl cluster-info
kubectl config get-contexts
kubectl config set-context --current --namespace=dev
```

Mini-lab

1. Write the Compose file above (use `nginx` and `redis` if you lack images) and bring it up. Read logs of one service, then bring it down.
2. Create a local cluster with kind or minikube.
3. Apply the Deployment, scale to 3, delete one pod, and watch it return with `kubectl get pods -w`.
4. Break it with an image tag that does not exist. Diagnose via `kubectl describe pod` events.
5. Fix it, check `kubectl logs`, and port-forward: `kubectl port-forward svc/hello 8080:80`.

Decision guide / failure modes

Need	Choose
Local dev, one machine	Compose
Single small server, simple stack	Compose or systemd + containers
Many nodes, self-healing, rolling updates	Kubernetes
Just logs from a pod	`kubectl logs`
Why is it not starting	`kubectl describe`, then events

Failure modes:

- `depends_on` does not wait for the database to be ready; add healthchecks or retry logic.

- Wrong kube context: you “delete” in production. Check `kubectl config current-context` first.
- Logs for a crashed container need `--previous`.
- Latest tags and no resource limits make clusters unpredictable.
- Secrets in plain YAML in git. Use a secret manager or sealed secrets.
- Treating pods as pets: they are replaceable; store state in volumes or external services.

Common mistakes

- Learning only kubectl YAML without Linux debugging
- exec as the only debug tool
- Running privileged pods casually

Exercises

Work in `exercises/37-compose-k8s-lite/`. Read that folder’s `README.md` fully before coding.

```
cd exercises/37-compose-k8s-lite
chmod +x test.sh exercise.sh
./test.sh
```

What you will build

Implement `kubectl_logs_cmd` in `exercise.sh` so it prints the two-word command prefix you use to read the standard output of a container in a pod. Print exactly `kubectl` followed by one space and the subcommand, with no pod name and no flags. When the test passes, practise on a local kind or minikube cluster: start a pod, read its logs, and also try the option that shows the logs of a previous crashed container. Keep the function body to one line; the goal is recall of the lesson vocabulary, not cleverness. Finish by re-reading the Decision guide table and noting one failure mode you could have caused yourself.

Study loop

- **A (Drill):** make `./test.sh` pass
- **B (Apply):** run 2–3 lab commands from this lesson on a real Linux host; paste notable output into your notes
- **C (Challenge):** rewrite the exercise from memory the next day

Do not open `solutions/37-compose-k8s-lite/` until you have an honest failing attempt.

Self-check

Try to answer without scrolling up:

1. Pod vs container?
2. How fetch logs?
3. Name a node-level Linux failure

Further reading

- https://kubernetes.io/docs/concepts/